Hemo Connect - Relatorio Tecnico DevSecOps Avancado

Status da entrega

Data local: 02/11/2026; os scanners registram tambem 03/11/2026 em UTC. Autores conforme documentacao: Marcos Zeller e Willian Donizetti; UMC. Consolidacao tecnica assistida por IA.

Implementacao e execucoes locais reais realizadas. Evidencias obrigatorias do GitHub Actions ainda pendentes. A entrega nao atende integralmente a rubrica enquanto os prints e artefatos publicados nao forem comprovados.

O responsavel determinou manter tudo local: nenhum commit, push, PR, merge ou deploy. Apesar de autorizar protecao de branch, os novos workflows nao estao publicados e seus checks nao existem no GitHub. Exigi-los agora criaria bloqueio sem check executavel; a protecao nao foi alterada.

Baselines locais: backend 7363db1455c191306c5af9a30a850d367478c535; frontend af546b038d0307635658cd463c475542e9531b84. As alteracoes desta atividade nao possuem commit.

1. Introducao a arquitetura open-source

O backend utiliza FastAPI, SQLAlchemy, PostgreSQL e Argon2id; o frontend, React, TypeScript, Vite e npm. A esteira existente de testes/build, Semgrep, TruffleHog e Trivy SCA foi preservada.

Cada repositorio recebe tres jobs independentes: ZAP DAST, Bearer e Trivy SBOM. O workflow executa em PRs para main, pushes em main e workflow_dispatch. Permissoes contents:read, Actions fixadas por commit, imagens por digest e binarios/regras por SHA-256 reduzem riscos da esteira. Timeouts, concorrencia e retencao de artefatos por 14 dias limitam recursos.

Fluxo previsto: codigo versionado -> runner Linux -> scanners -> relatorios e SBOM -> artefatos para download -> checks exigidos na protecao de branch. O YAML nao substitui a configuracao desses checks nem comprova uma execucao.

ZAP e Trivy sao open-source e nao exigem servico pago ou chave de API. Ressalva de licenciamento: Bearer 2.1.3 e seu pacote de regras utilizam Elastic License 2.0, verificada nas fontes oficiais. E uso gratuito para esta execucao interna, mas source-available com restricoes, nao uma licenca open-source aprovada pela OSI. Bearer foi mantido porque e exigido pelo enunciado; o requisito literal "100% open-source" precisa ser alinhado com o docente antes do aceite. Nao foi substituida a ferramenta sem autorizacao. GitHub Actions e uma plataforma hospedada, nao integralmente open-source. Runners padrao em repositorios publicos sao gratuitos; em privados, e necessario respeitar a franquia. Os testes nao usam dados reais, Brevo, Supabase ou API implantada.

2. Descritivo tecnico das ferramentas

2.1 OWASP ZAP - DAST

Foi executado ZAP 2.17.0, Java 17.0.21, com imagem linux/amd64 fixada no digest 0075662e5c287cf0a68caebabbfea872e5dc461912ad6d7345fa02a290d0ccb6. O Docker local tornou-se disponivel apos inicializacao suportada; a indisponibilidade anterior nao foi tratada como scan aprovado.

O backend usa PostgreSQL 16 descartavel, factory exclusiva de CI, contas sinteticas de doador/administrador e adaptador local de e-mail. A factory recusa banco remoto, nome/usuario diferente de hemo_dast, ambiente nao CI e banco previamente populado. Login por senha e 2FA reais geram sessoes.

Corrigiu-se um defeito da fixture: enderecos .test eram recusados pelo validador real de LoginRequest. Agora sao usados doador@example.com e administrador@example.com, do dominio reservado example.com. O adaptador aceita somente esses destinatarios e nao envia mensagens pela rede. Um teste de regressao verifica os enderecos no modelo real.

A preparacao comprova 10 leituras autenticadas entre os dois perfis. O ZAP utiliza OpenAPI filtrado, requestor com status 200, scan ativo e espera passiva. O escopo GET inclui auth/me, usuarios, hemocentros, hemocentros/1 e, para doador, agendamentos/me e historico/me. Mutacoes destrutivas, logout e jornadas clinicas completas nao sao cobertos.

O frontend usa bundle local, requestor, spider e spiderAjax, com scan passivo; nao cobre jornadas autenticadas completas nem scan ativo da SPA. A API do build aponta para https://localhost:8000, nao para a implantacao. No Windows, o scanner acessa os servicos via host.docker.internal; o preview recebe allowlist explicita desse host. No CI Linux, utiliza-se loopback e rede host. Essa adaptacao nao altera o deploy.

Tokens/planos privados nao sao artefatos; JSON/HTML sao sanitizados. O ZAP bloqueia High, sua maior severidade, com exit-code 1. Erros/warnings reportados pelo Automation Framework, falha de execucao e relatorio ausente/invalido tambem bloqueiam. Os insights do relatorio nao sao equivalentes a warnings do Automation Framework.

Resultados: frontend terminou com codigo 0, contendo 0 High, 2 Medium, 2 Low e 1 Informational, por classe de alerta. Doador e administrador terminaram com codigo 0 e nenhuma classe de alerta. Isso significa ausencia de High detectado nesse escopo, nao ausencia de defeitos ou cobertura integral.

Degradacao observada e nao encerrada: o relatorio do doador registra 11 warnings internos, 8% de falhas de rede e 6% de respostas 5xx (68% 2xx e 25% 4xx; percentuais arredondados pelo ZAP). O log real da API mostrou 500 ao consultar hemocentros com identificadores maiores que a faixa INTEGER do PostgreSQL, com NumericValueOutOfRange. O erro tambem apareceu durante o scan do administrador. E um achado preexistente de validacao de entrada, nao uma falha corrigida da fixture. Nao foi introduzida supressao nem alteracao ampla das rotas para esconder o resultado. As falhas de rede permanecem sem causa individual comprovada. Os scans completaram, mas essa degradacao limita a confiabilidade da cobertura; exige triagem adicional antes de declarar DAST integralmente aceito.

Foi realizado tambem um teste real contra alvo local indisponivel: ZAP retornou 2 e o gate da aplicacao retornou 1, recusando relatorio ausente. Nao se interpretou ausencia de achados como sucesso quando nao houve scan.

2.2 Bearer - seguranca e privacidade

Bearer 2.1.3 analisa app no backend e src no frontend. O workflow utiliza fail-on-severity critical e exit-code 1; as outras severidades continuam no relatorio. Falhas de ferramenta e relatorio ausente/invalido nao sao convertidos em sucesso.

O binario nao incorpora o pacote de regras. A tentativa offline sem regras falhou explicitamente; o instalador passou a baixar o pacote oficial completo, incluindo licenca, na revisao 82c335f781a61a6d8b052394f84fd1d93edec467. Binario e pacote sao verificados por SHA-256. disable-default-rules com external-rule-dir substitui o download dinamico pelo pacote completo fixado; nao desativa as regras de seguranca. no-extract reduz trechos de codigo nos artefatos.

Backend: codigo 0, sem Critical e com 4 Medium: python_lang_exception em app/routes/auth.py:53 e python_lang_logger em app/services/auth.py nas linhas 192, 408 e 424. Esses sinais exigem triagem de erros/logs e possivel exposicao de dados; nao foram declarados exploits confirmados.

Frontend: a primeira execucao retornou 1 por dois Critical de javascript_lang_hardcoded_secret. A verificacao direta mostrou strings de rotas publicas em src/routes.ts:4-5: /auth/forgot-password e /reset-password, nao credenciais. O arquivo . security/bearer.ignore, produzido pela CLI, registra apenas os fingerprints c48985ef6e419bead72b63c9ba1a2a39_0 e c48985ef6e419bead72b63c9ba1a2a39_1, com justificativa e reavaliacao em 02/12/2026. A regra segue ativa. O scan final retornou 0, sem achados nao suprimidos; os relatorios anteriores foram preservados.

Gate real comprovado: uma fixture separada, nunca executada ou integrada ao produto, contendo string explicitamente sintetica em password foi escaneada com o mesmo pacote e arquivo de excecoes. Produziu Critical e exit-code 1. Nao representa segredo real vazado nem PR bloqueado.

Os dois relatorios JSON de privacidade foram gerados com codigo 0. Backend: 4 entradas de titular, com 53, 1, 1 e 1 deteccoes; frontend: 2 entradas, com 11 e 1 deteccoes. subject_name e Unknown nessas entradas: a atribuicao automatica ao titular e limitada, devendo ser revisada manualmente. Deteccoes nao equivalem a pessoas, campos unicos ou incidentes. FastAPI/SQLAlchemy nao possuem cobertura integral garantida; nao se presumem recursos Pro nem certificacao juridica de conformidade LGPD.

2.3 Trivy - SBOM e Supply Chain

Trivy 0.76.1 gerou sbom.json reais em CycloneDX 1.7: backend com 35 componentes totais e frontend com 276, incluindo desenvolvimento. O gate verifica formato, inventario nao vazio, versoes de dependencias e pacotes esperados. O componente application do manifest pode nao ter versao; nao e contado como biblioteca resolvida.

requirements.txt do backend nao fixa versoes. Um ambiente virtual limpo, com somente dependencias do aplicativo, gerou pip freeze com transitivas. O snapshot local e Windows/Python 3.12; o CI Linux gera seu proprio inventario. Nao se afirma igualdade com deploy ou reproducibilidade total. No frontend foi utilizado package-lock.json com include-dev-deps.

CycloneDX nao executa scanner de CVEs por padrao. O Trivy SCA existente, com HIGH/CRITICAL, foi preservado separadamente. Informacoes de licenca podem ser incompletas sem site-packages/node_modules no alvo; o SBOM nao certifica ausencia de CVEs ou regularidade de licencas.

3. Integracao com os requisitos, LGPD e Triade CIA

Requisitos originais	Contribuicao	Limite
1.5-1.11 e 2.1-2.7	Contextos autenticados, senha/2FA reais, DAST e regras estaticas.	Nao substitui revogacao, titularidade, recuperacao e concorrencia.
3.1-3.8	Headers/trafego e regras de criptografia.	HTTP local nao comprova TLS ou cifragem em producao.
4.1-4.3	Relatorios reais de privacidade e inventario manual.	Titulares Unknown e limitacoes de framework exigem revisao humana.
4.4-4.11	Mapeamento de consentimentos/direitos.	Revogacao, retencao e exclusao necessitam evidencia funcional.
5.1-5.4	Achados reais de erros/logs.	Nao comprova append-only ou integridade concorrente.
6.6-6.12	SBOMs reais, gates negativos e rastreabilidade.	Evidencias Actions e protecao de merge ainda pendentes.

Confidencialidade: sessoes reais e busca por exposicao de dados. Integridade: gates, checksums, versoes e artefatos verificaveis. Disponibilidade: execucao isolada, limites e registro explicito dos 500 e falhas de rede, sem ataques contra a implantacao.

Dados/fluxo	Tratamento observado	Ponto a comprovar
Nome, CPF, e-mail e telefone	Cadastro, perfil, banco e exportacao.	Finalidade/base legal, acesso minimo e retencao.
COREN/UF	Solicitacao e aprovacao profissional.	Retencao e evidencia da conferencia.
Saude, pre-triagem, sangue e resultado	Reservas, atendimento e triagem.	Classificacao de dados sensiveis, necessidade e conservacao.
Consentimentos e versao	Registro por finalidade e revogacao.	Catalogo validado e efeito sobre tratamento.
E-mail em logs	Auditoria e operacao.	Pseudonimizacao, acesso, integridade e retencao.
Brevo e PDF	Mensagens e exercicio de direitos.	Destinatarios, copias e exclusao/anonimizacao.

Inventario documental/pontual, nao auditoria juridica completa. A adicao de scanners nao encerra pendencias operacionais da release CIA anterior.

4. Validacoes locais

Verificacao	Resultado
actionlint 1.7.12 nos dois workflows	Aprovado.
Gates/preparacao backend	11 testes aprovados, incluindo validacao real dos e-mails sinteticos.
Gates frontend	6 testes aprovados.
Regressao auth/usuarios/proxy	74 testes aprovados; 6 avisos preexistentes.
Frontend lint/build	Aprovados; aviso preexistente de bundle aproximadamente 1,08 MB.
ZAP frontend e dois perfis backend	Execucoes reais concluidas, gates aprovados; limitacoes e degradacao descritas em 2.1.
Bearer security/privacy nos dois repositorios	Execucoes reais concluidas; 4 Medium backend e duas excecoes pontuais frontend.
Bearer Critical sintetico	Achado Critical real do scanner e exit-code 1.
ZAP alvo indisponivel	Scanner exit 2, gate exit 1; nenhuma aprovacao ficticia.
SBOMs e gates reais	Backend 35; frontend 276 componentes.
Helpers compartilhados	Gates e instaladores identicos nos dois repositorios.
Shell Linux	Arquivos .sh fixados como LF via .gitattributes; validacao do instalador na imagem Linux.
GitHub Actions/protecao de merge	Nao alterados/executados; novos workflows nao publicados.

Testes sinteticos adicionais verificam ZAP High, JSON invalido/ausente, codigo de falha e SBOM sem versoes/pacotes. Esses testes nao sao incidentes.

5. Evidencias reais e rubrica

Arquivos em evidencias/devsecops-avancado:

- sbom-backend/sbom.json e inventory/requirements.txt.

- sbom-frontend/sbom.json.

- zap-frontend/zap-frontend.json e .html.

- zap-backend/zap-doador.json/.html e zap-administrador.json/.html.

- zap-backend/coverage.json e integer-boundary-errors.txt, com leituras reais

e trechos nao sensiveis do log de erro da API.

- bearer-backend/bearer.json e privacy.json.

- bearer-frontend/bearer.json, privacy.json e relatorios antes da triagem.

- gates/bearer-critical-fixture.json e gates/zap-unavailable.log.

Prints de execucao LOCAL, nao do GitHub Actions

As imagens abaixo sao recortes de capturas reais dos relatorios HTML locais do ZAP, para legibilidade, sem alterar o conteudo apresentado. Nao mostram jobs do Actions, upload ou download remoto.



ZAP by Checkmarx Scanning Report

Site: http://host.docker.internal:4173

Generated on Tue, 3 Nov 2026 02:15:27

ZAP Version: 2.17.0

ZAP by Checkmarx

Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	2
Low	2
Informational	1

ZAP frontend local: relatorio real, com achados nao criticos.

Level	Reason	Site	Description	Statistic
Low	Warning		ZAP warnings logged - see the zap.log file for details	11
Low	Exceeded Low		Percentage of network failures	8 %
Info	Informational	http://host.docker.internal:8000	Percentage of responses with status code 2xx	68 %
Info	Informational	http://host.docker.internal:8000	Percentage of responses with status code 4xx	25 %
Info	Informational	http://host.docker.internal:8000	Percentage of responses with status code 5xx	6 %
Info	Informational	http://host.docker.internal:8000	Percentage of endpoints with content type application/json	100 %
Info	Informational	http://host.docker.internal:8000	Percentage of endpoints with method GET	100 %
Info	Informational	http://host.docker.internal:8000	Count of total endpoints	6
Info	Informational	http://host.docker.internal:8000	Percentage of slow responses	6 %

ZAP backend local, perfil doador: insights incluem falhas de rede e respostas 5xx; resultado nao equivale a aplicacao sem defeitos.

Criterio	Peso	Situacao
Pipeline app/ZAP/Bearer/Trivy	35%	Implementado e scanners locais executados; execucao Actions e triagem de degradacao DAST pendentes.
Artefato e bloqueio	25%	SBOMs e gates locais reais; upload/download e checks obrigatorios no GitHub pendentes.
Relatorio e prints	40%	PDF atualizado com capturas locais; prints obrigatorios do Actions e artefato publicado ausentes.

Nao foram fabricados jobs verdes, downloads GitHub ou PR bloqueado. Nao se atribui nota ou aceite integral a evidencias apenas locais.

6. Etapas ainda em aberto

1. Alinhar com o docente a ressalva ELv2 do Bearer. Triar os identificadores

fora da faixa INTEGER e as falhas de rede dos scans backend; aplicar correcao especifica com teste PostgreSQL quando autorizada e repetir os scans afetados. Revisar Medium de ZAP e Bearer e os titulares Unknown, sem desativar regras para obter verde.

2. Mediante autorizacao para publicacao, revisar e versionar as alteracoes

dos dois repositorios; publicar em branch apropriada sem merge/deploy automatico. A decisao atual proibe essa etapa.

3. Executar workflow_dispatch ou PR para main com Python 3.12, Node 22

e Docker no runner; confirmar os tres jobs e todos os artefatos.

4. Somente apos checks existentes e executados, exigir os tres novos

checks junto aos atuais na protecao de branch. Confirmar bloqueio de merge quando falham; ligar deploy aos checks se houver autorizacao.

5. Baixar sbom-backend e sbom-frontend no Actions e validar sbom.json.

6. Capturar os tres jobs verdes e o artefato para download em cada

repositorio, com URL, data e commit; inserir no PDF final.

7. Referencias

Consulta em 02/11/2026.

- OWASP ZAP Automation Framework: https://www.zaproxy.org/docs/automate/automation-framework/

- OWASP ZAP exitStatus: https://www.zaproxy.org/docs/desktop/addons/automation-framework/job-exitstatus/

- OWASP ZAP OpenAPI: https://www.zaproxy.org/docs/desktop/addons/openapi-support/automation/

- Bearer CLI: https://docs.bearer.com/reference/commands/

- Bearer linguagens: https://docs.bearer.com/reference/supported-languages/

- Licenca Bearer 2.1.3: https://github.com/Bearer/bearer/blob/v2.1.3/LICENSE.txt

- Elastic, licencas nao OSI e alternativas: https://www.elastic.co/pricing/faq/licensing

- Pacote de regras fixado: https://github.com/Bearer/bearer-rules/tree/82c335f781a61a6d8b052394f84fd1d93edec467

- Trivy Python: https://trivy.dev/docs/latest/coverage/language/python/

- Trivy filesystem: https://trivy.dev/docs/latest/target/filesystem/

- GitHub Actions custos: https://docs.github.com/en/actions/concepts/billing-and-usage

- LGPD, Lei 13.709/2018: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm